Beta-Testing Without Beta-Testers: Agent-Driven Closed-Loop Game-Design Optimisation for Monopoly

Selim Emir Can (selimcan@stanford.edu)

2026-06-06

Abstract

Balancing a game traditionally demands many human testers across many sessions, making design iteration slow and expensive. We present a closed-loop, agent-driven workflow that automates the inner loop of playtesting: a genetic algorithm searches a 66-dim Monopoly board-design space against a composite objective (fairness, length, draw rate, money transfer), scoring each candidate against a fixed pool of 30 diverse rule-based strategies, with an instruction-tuned LLM (Qwen2.5-1.5B) as an independent cross-class probe. The optimised board improves the composite by ∼47% over the default at 2 and 3 players, and three results validate it: **(1)** the 2p/3p-trained boards keep a ∼2× advantage at every count from 2 to 8, except fairness, which is regime-specific; **(2)** both evaluator classes prefer the pool-driven board over an LLM-trained one on 4 of 5 metrics, so a diverse pool generalises across agent classes at zero per-decision LLM cost; and **(3)** in a human pilot (N=10, two players) all trained effects survive, with humans showing a *larger* default-board failure than either simulator, so the agent predictions are conservative, not optimistic. Beyond Monopoly, the recipe transfers to any parameterised multi-agent system: express balance as a small measurable objective, score designs against a deliberately diverse pool of cheap imperfect agents (which exposes blind spots a single strong agent hides), and cross-check the winner with an independent agent class before trusting it. Code, logs, and figures: https://github.com/Selim-Emir-Can/CS348K-proj/tree/master.

1 Background and setup

The problem. Game designers face combinatorial design spaces that human playtesting cannot cover. Tabletop Monopoly is a sharp testbed for the agent-assisted alternative: a small but rich parameter space and well-known balance issues (games drag on, single-property luck dominates, two-player play is unbalanced). The goal of this project is not to "solve" Monopoly but to demonstrate a reproducible designer-facing beta-testing loop:

> **Parameterise environment → evaluate against a diverse agent population → compare agreement across agent classes → shortlist candidate designs → send only the most informative cases to human playtest.**

Inputs and outputs. The pipeline takes a board configuration $\theta \in \mathbb{R}^{44} \times \{0,1\}^{22}$ (22 cost multipliers, 22 rent multipliers, 22 keep-mask bits) and produces (i) a scalar composite score plus a per-component metric tuple $(\overline{F}, F_{\max}, \overline{R}, \overline{D}, \overline{T})$ estimated over 100 games against the strategy pool, and (ii) a decision-quality artefact (per-strategy heatmap, per-game JSONL, or per-decision LLM log) that lets the designer audit *why* θ scored as it did.

Goals and constraints. What makes a game of Monopoly *fun*? We take a good game to be one that (i) stays competitive, with every player keeping a plausible path to victory rather than one seat or strategy running away with it; (ii) fits a single sitting, neither dragging on long after the outcome is clear nor ending before choices matter; (iii) reaches a decisive winner instead of stalemating to a time-out; and (iv) keeps money and property actively changing hands, so turns stay tense rather than passive accumulation. **We chose the objective's metrics to make each of these qualities measurable: fairness (mean and**

worst-pair) for competitiveness, length for sitting-time, draw rate for decisiveness, and money transfer for liveliness. Optimising their weighted sum, rather than any single proxy, forces "fun" to be a balance of all four at once. Concretely, the optimiser minimises a weighted sum of five design qualities (Eq. 1), chosen so that no single term dominates: mean pair-fairness $\overline{F}$, worst-pair fairness $F_{\max}$, length deviation $|\overline{R} - R^*|/R^*$, mean draw rate $\overline{D}$, and player-to-player money-transfer deviation $|\overline{T} - T^*|/T^*$. Hard constraints: every candidate runs end-to-end at the per-property cost/rent bounds $[0.5, 2.0]$, retains both utilities and railroads, and finishes or truncates within 200 rounds. Soft constraints: total wall-clock budget under 30 minutes on a single CPU (≈ 6 ms/game) for the rule-based pipeline, and under 6 hours on a single 12 GB GPU for the LLM probe.

$$\text{score}(\theta) \;=\; w_{\text{fair}}\overline{F} \;+\; w_{\text{fmax}}F_{\max} \;+\; w_{\text{len}}\tfrac{|\overline{R}-R^*|}{R^*} \;+\; w_{\text{draw}}\overline{D} \;+\; w_{\text{money}}\tfrac{|\overline{T}-T^*|}{T^*} \tag{1}$$

Defaults: $w = (1, 0.5, 0.5, 0.3, 0.3)$, $R^* = 60$ rounds, $T^* = 100$ \$/round. The GA optimises against $T^* = 100$ \$/round at 2p / 3p training counts; for cross-count reporting (Section 4.7) we use the normalised metric $\overline{T}_{\text{pt}} = \overline{T} \,/\, n_{\text{players}}$ with $T^*_{\text{pt}} = 50$, which equals the original training target at 2p and removes the trivial linear scaling of money flow with seats so cross-count comparison is meaningful.

Hypothesis. The falsifiable claim of the project is two-fold: (a) a candidate board's directional ranking under a 30-strategy parametric pool matches its directional ranking under an independent agent class (a 1.5B-parameter LLM with a deterministic per-field state validator); and (b) the same directional ranking also holds in real human play, so the strategy pool acts as a proxy for human behaviour on the design axes the optimiser targets. (b) is the claim required to use agent feedback as a beta-testing surrogate.

The crux. The hard part is the question of *what evaluates a candidate board* without human playtesters. A single trained agent collapses "design quality" to one playstyle's quirks; the honest alternative is a *population* of strategies, which raises the real question: *which strategies, and how many.* This cannot be shortcut by intuition. Even a seasoned player cannot zero-shot a balanced board or fix the default one with plausible hand-tweaks (dropping properties to shorten the lap, raising rents to force bankruptcies): the 66 interacting knobs trade the five objectives off against one another, so a rent hike that shortens games can hand the win to one archetype, and fairness depends on how *many playstyles* fare against each other, which no designer can introspect. Pinning this down requires playing the games out, which is the point of the project: a population of agents as the evaluator surfaces design insight that intuition and trivial edits cannot.

2 Related work

Automated playtesting and agent-based design evaluation. Procedural and search-based content generation has a long literature [8], and authoring tools such as Sentient Sketchbook [5] use search agents to surface design issues. Closest to our work is the line on *procedural personas* [3], which represents distinct player types as evolved or trained agents and uses them both to playtest and to drive content generation; optimising a design against such an agent population is established prior art. Small instruction-tuned LLMs such as Qwen2.5 [7] have likewise served as decision-making agents, and generative agents [6] treat LLMs as populations of social actors. Our contribution is not any single evaluator but a *protocol that cross-checks multiple agent classes*: a fixed parametric rule-based pool and an independent LLM probe, with disagreement-as-diagnostic and human playtesting reserved for boundary cases. The pool uses fixed hand-specified and sampled rule-based parameters rather than trained policies, since per-candidate model-free RL is impractical for a high-variance stochastic game evaluated over thousands of designs.

Tree search and evolutionary search. Browne et al. [1] survey Monte Carlo tree search, and population-based competitive evaluation has been used at scale for StarCraft II leagues [10]; we reuse the "population as evaluator" intuition in inverted form, fixing a diverse population and searching over the environment instead. The search itself is a standard GA [2] used unchanged.

3 Approach

The workflow is built around the game designer: they specify the objective weights and targets (Eq. 1) and the strategy pool, the GA proposes board designs, the simulator scores each candidate against the pool, and the cross-class checks together with the playtest pilot indicate which designs are worth taking to real users (Figure 1). The strategy pool is a fixed set of 30 diverse rule-based strategies (10 hand-designed archetypes, listed in Table 1, plus 20 sampled from the parametric space), chosen so that worst-case fairness across the pool is a meaningful objective. The system is implemented on top of a single-process Monopoly core game loop and pretrained Qwen2.5-1.5B-Instruct [7] weights.

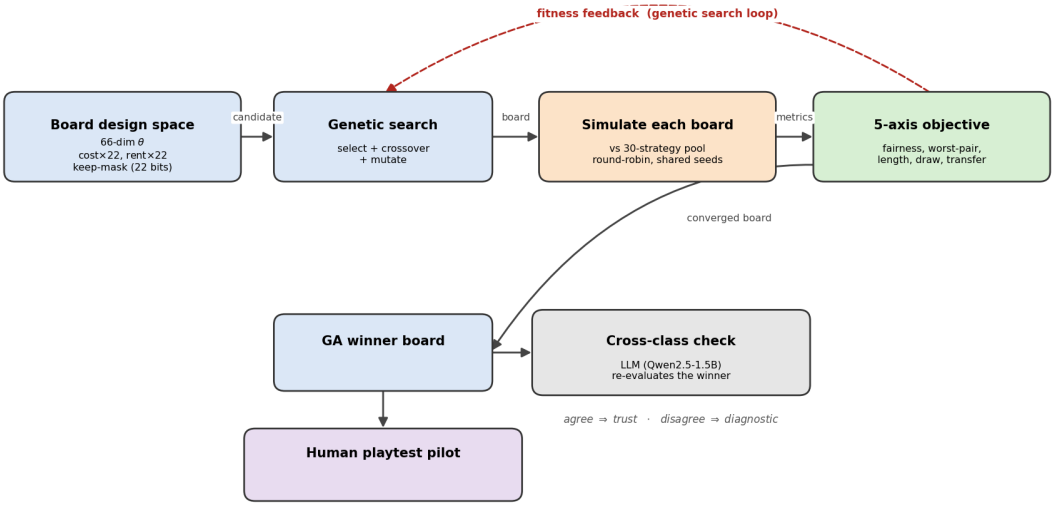


Figure 1: **Closed-loop game-design optimisation.** The GA proposes a board vector θ; the simulator evaluates each candidate against a fixed 30-strategy rule-based pool at shared seeds; the composite score and per-component metrics are returned to the GA. The same winning board is then re-scored under an independent agent class (all-LLM seats with structured echo validation) and against real human play in a small pilot. The designer controls the objective weights and the pool composition; everything downstream is deterministic given a seed.

Strategy	Cash floor	Trades	Build	Buys	Ignores groups
AggressiveBuilder	0	no	max	RR+Util	none
CashHoarder	1000	no	slow	RR+Util	none
Trader	300	yes	max	RR+Util	none
RailroadKing	100	no	max	RR only	all colour groups
LowCostOnly	200	yes	max	RR+Util	5 priciest (Or/Rd/Yl/Gn/In)
HighCostOnly	400	yes	max	RR only	3 cheapest (Br/Lb/Pk)
Balanced	300	yes	max	RR+Util	none
Passive	500	no	slow	none	none
Bully	50	yes	max	RR+Util	none
RiskAverse	800	no	slow	RR only	Green, Indigo

Table 1: **The 10 hand-designed strategy archetypes.** "Cash floor" is the cash kept un-spent; "Build" is max houses/turn vs. one/turn; "Buys" is railroads / utilities. The pool adds 20 strategies sampled once from the same parametric space.

The implementation is laid out as a standard optimiser/simulator split. The rule-based stack is a 17-knob parametric player, a 66-dim design-space encoder, a thin in-loop simulator that wraps the upstream Monopoly

core game loop to return per-game stat dicts, and a GA with a matched-budget random-search baseline. A module-by-module breakdown and the canonical driver scripts are in Appendix A.12; the LLMPlayer prompt and validator are spelled out in Appendix A.7.

The key evaluation decision is that the 30-strategy pool, rather than any single agent, is the evaluator: a single agent would collapse design quality to its own quirks, whereas the pool makes worst-case fairness a first-class objective.

What did not work. Two iterations are worth recording. Qwen2.5-0.5B-Instruct as the LLM probe was uninformative; it emitted near-uniform `BUY` regardless of cash or opponent-completion state, so smaller-than-1.5B parameters did not clear the bar for the cross-class probe to be meaningful. The v1 prompt with a free-form validator induced retry-fabrication (47 retry-induced hallucinations across 2,304 calls) before we realised the issue was a type-coerced cash field in the validator, not the model; v2's structured ECHO block plus deterministic per-field validation drives this to zero across 2,207 calls (Appendix A.10).

4 Evaluation and results

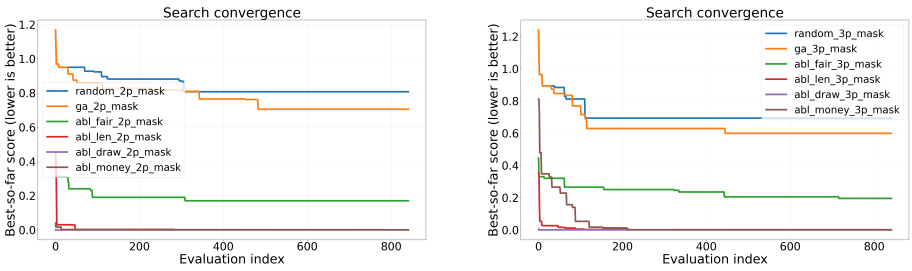
Definition of success. The project is successful if all four of the following hold:

1. the GA finds a board that improves the composite score over the default by a margin larger than the noise floor, verified at the optimum with n=1000 games;
2. an independent agent class reproduces the same directional ranking on the optimised boards;
3. the predicted effects survive contact with real human play, in the same *direction* and similar *relative magnitude* as under the agent evaluators;
4. the strategy pool is justified as the evaluator: running the GA with the LLM as evaluator instead should still converge and beat the default, yet the resulting LLM-driven board should lose to the rule-based GA winner under both evaluator classes (Section 4.3).

Experimental setup. For each player count $n \in \{2, 3\}$ we draw 10 evaluation matchups from the pool under diversity constraints: every archetype appears in at least one matchup, and at least 3 matchups include a sampled strategy. Compute budget and total game counts are reported in Appendix A.13.

4.1 Default-board baseline and search

The default board is the identity vector under the encoder. It scores 1.463 on the composite at 2p and 1.230 at 3p; these are the reference points every optimisation result is compared against. Per-metric numerical results for the default and the GA winners across player counts are reported in Table 2.

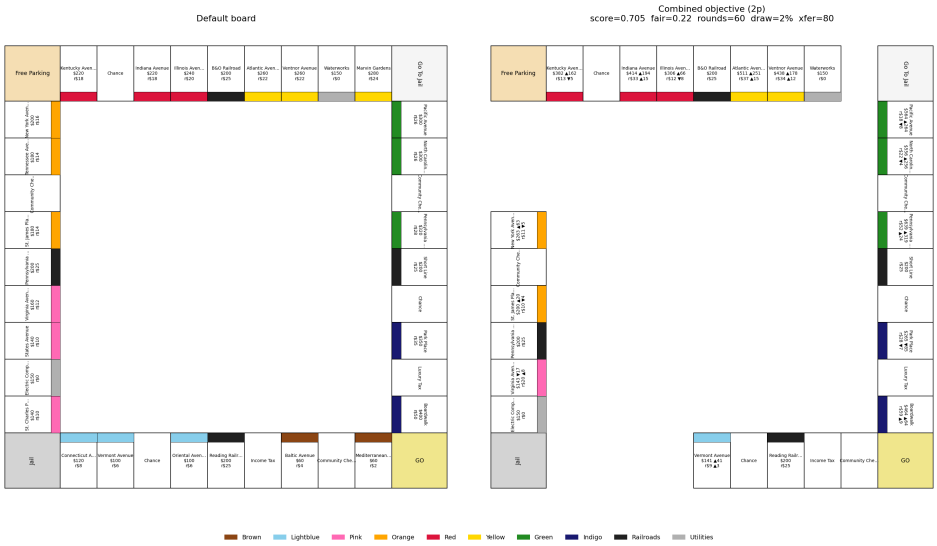


(a) 2 players (b) 3 players

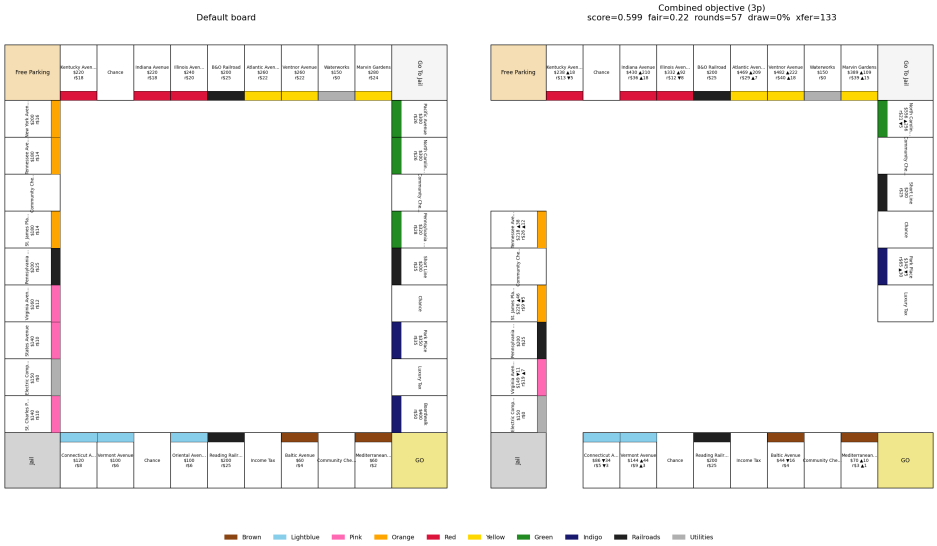
Figure 2: **The GA beats random search by ∼13% at matched budget** (best-so-far composite, lower is better). At 2p 0.705 vs. 0.807; at 3p 0.599 vs. 0.694. The lower curves are single-objective ablations; the ∼47% lift over the default dwarfs the GA-vs-random gap.

The larger lift comes from problem framing rather than optimiser cleverness (Figure 2): the composite objective, the fixed strategy pool, and shared random numbers make random search already a strong baseline, so the bottleneck is formulation, not search.

What does the GA actually build? The combined-objective GA winners (Figure 3) shrink the lap and rebalance costs and rents: the 2p winner drops 10 of the 40 cells via the keep-mask, retains both railroads and utilities, and re-tunes the remaining colour groups. The 3p winner keeps a different subset, since each player count converges to a different physical board.



(a) 2-player combined-objective GA winner.



(b) 3-player combined-objective GA winner.

Figure 3: **The boards the GA actually builds.** Each panel shows the default Monopoly board (left, 40 cells) alongside the GA winner (right, shrunk by the keep-mask), with per-property cost/rent multipliers and keep/drop annotations on the optimised side. The 2p and 3p winners are visibly different physical boards, consistent with the cross-evaluation finding (Section 4.7) that each design specialises to its training regime.

Single-objective ablations confirm the composite objective is non-redundant: setting each weight to 1 in turn drives the optimiser to a visibly different board, so no two terms are redundant (Appendix A.4).

4.2 LLM cross-class agreement

As an independent cross-class check, we re-evaluate the GA winners with all-LLM seats (Qwen2.5-1.5B-Instruct) and compare the per-metric change against the rule-based pool. Both evaluator classes agree on the direction of every metric for the default→GA-winner comparison, at both 2p (Figure 4) and 3p (Figure 5), with magnitudes matching closely on the composite, game length, and money transfer.

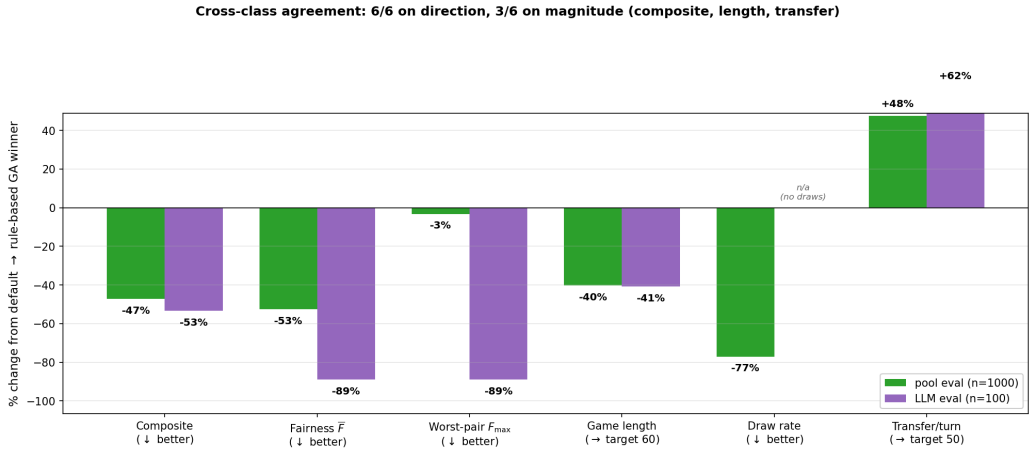


Figure 4: **Both evaluator classes agree on direction for all 6 metrics (rule-based GA winner vs. default, 2-player).** Per-metric % change under the 30-strategy pool (green, n=1000) and all-LLM seats (purple, n=100). Magnitudes match closely on composite ($-47\%/-53\%$), length ($-40\%/-41\%$), and transfer per player-turn ($+48\%/+62\%$); the two fairness terms agree in direction but diverge in magnitude (smaller LLM tail at n=100). Draw rate is in the panel footer because LLM seats produce no draws on either board.

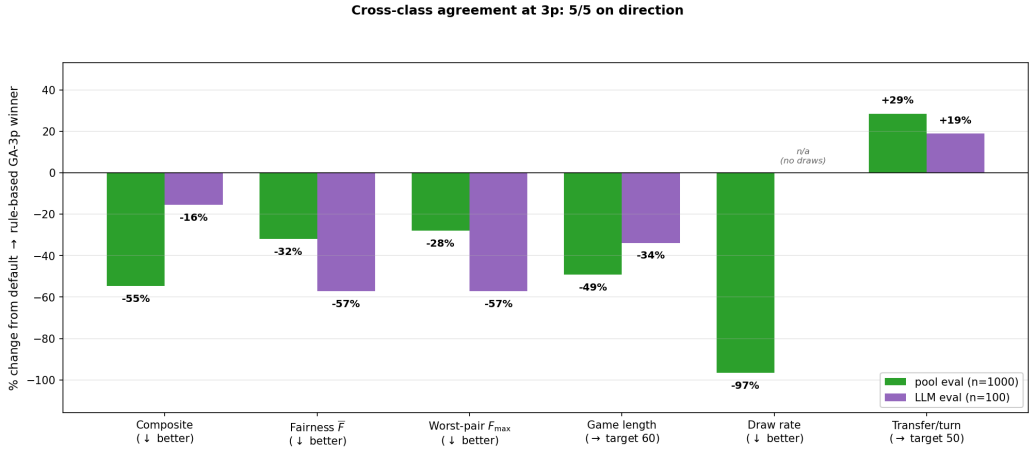


Figure 5: **Cross-class agreement at 3p: same-direction improvement on every measured metric.** % change default → GA-3p winner under pool (green, n=1000) and LLM seats (purple, n=100). Direction agrees on 5/5; magnitudes match on length ($-49\%/-34\%$) and transfer per player-turn ($+29\%/+19\%$). The 3p audit reproduces the 2p result (Figure 4).

4.3 LLM-driven GA and the cross-class verdict

We re-ran the GA with the LLM as evaluator (population 8, 5 generations, 5 seeds, 2-player only). The LLM-driven loop converges and beats the default under all-LLM evaluation (Appendix A.8). Cross-evaluating

that board against the rule-based GA winner under both agent classes (Figure 6) shows both classes prefer the rule-based winner on 4 of 5 stable metrics (composite, $\overline{F}$, $F_{\max}$, game length); the LLM-driven board wins only on transfer per player-turn, the metric its small-n training overfit to.

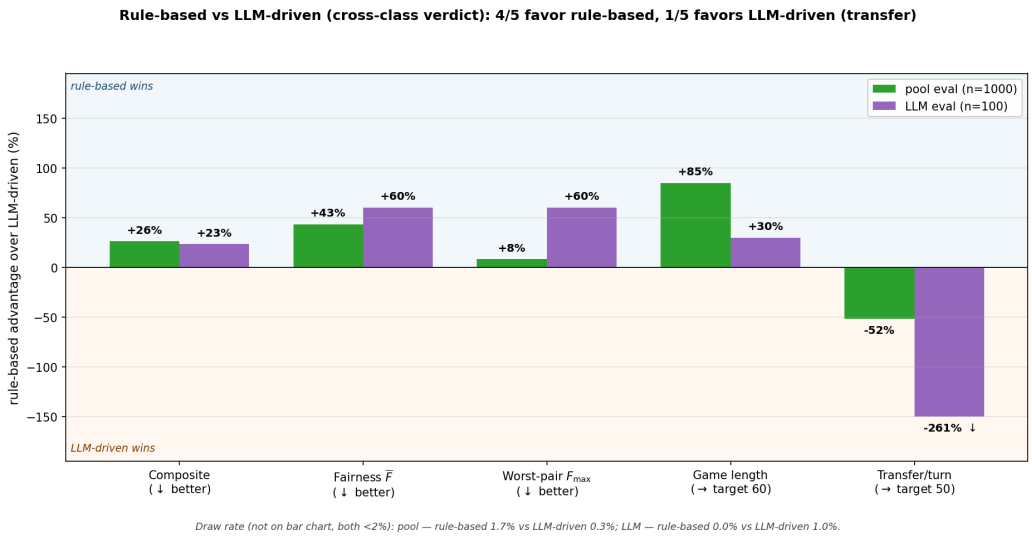


Figure 6: **Both evaluator classes deliver the same verdict: rule-based wins 4/5, losing only on transfer (2p).** Per-metric % advantage of the rule-based GA winner over the LLM-driven board under the pool (green, n=1000) and LLM seats (purple, n=100); positive = rule-based wins. Draw rate in footer (both $< 2\%$); the LLM transfer bar is clamped at -150% (true -261%).

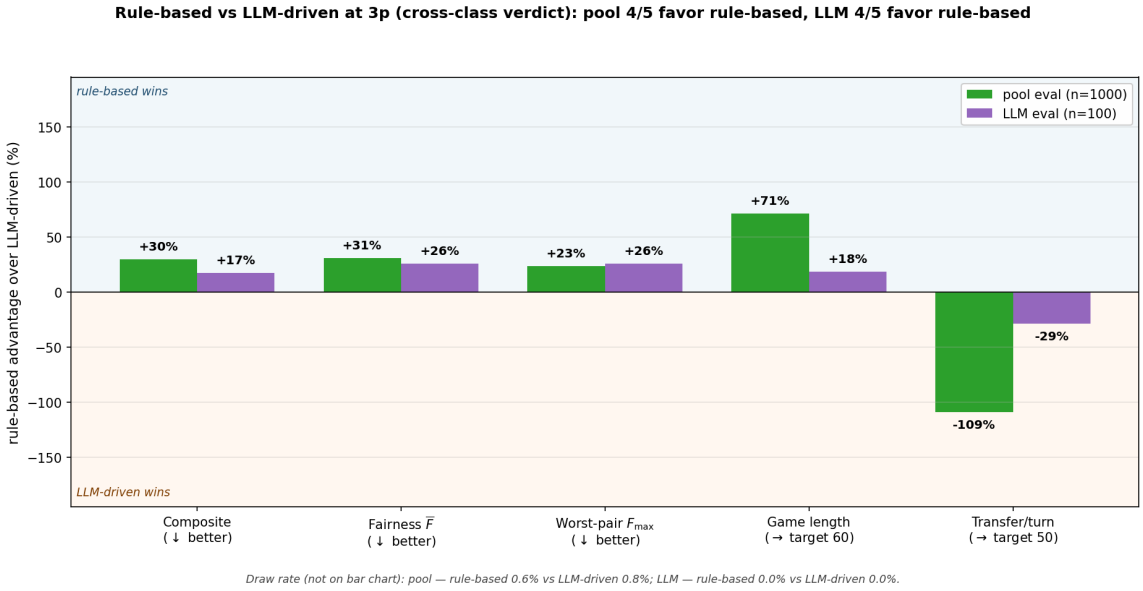


Figure 7: **Cross-class verdict at 3p: same 4/5 pattern as 2p.** Per-metric % advantage of the rule-based GA-3p winner over the LLM-driven board (n=100 LLM seats). Draw rates in footer (both $< 1\%$).

The interpretation: a sufficiently large rule-based pool produces a design that generalises to an evaluator class never seen at training (LLM seats), at zero per-decision LLM cost, whereas the LLM evaluator's narrow decision distribution makes its own optimiser overfit. A single LLM or RL agent as evaluator is a benchmark; a diverse strategy pool is a beta-testing surrogate. The 3p audits (Figures 5, 7) confirm the verdict holds

when the GA winner is itself a 3-player design. This check showed the strategy pool to be a better and cheaper alternative to LLM-based players as the evaluator, and motivated the human pilot (Section 4.4) as the stronger test of whether the design holds up with real people.

4.4 Human playtest pilot (N=10, two players)

As a reality check, two players played the default and the GA-2p combined winner (Section 4.1) ten times each at matched seeds, so a difference reflects the board rather than luck. We measured the four optimised metrics in each game (Figure 8).

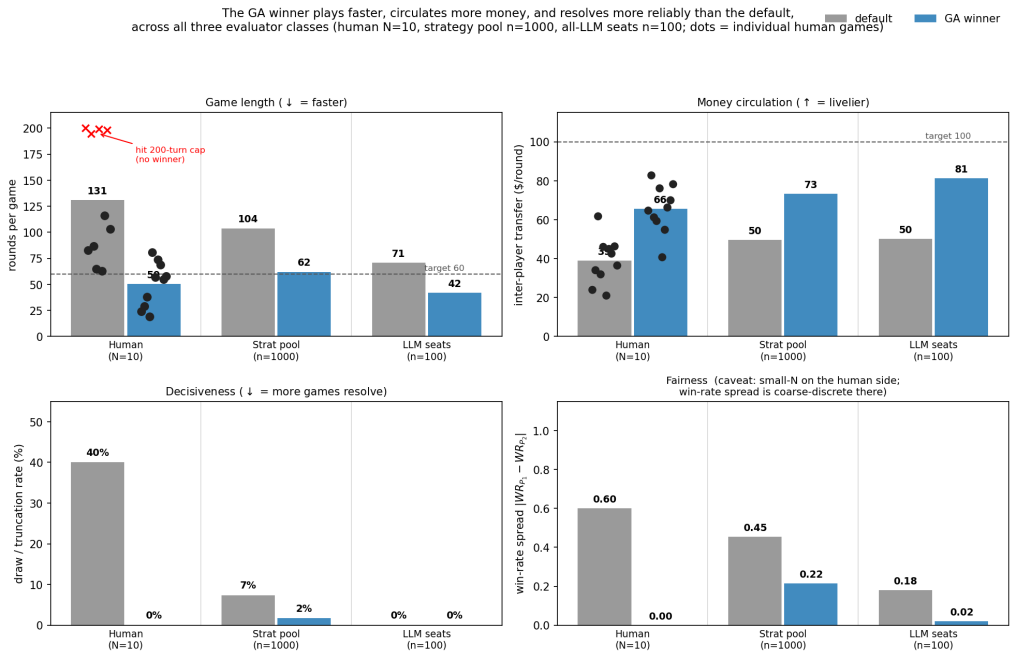


Figure 8: **The GA winner plays faster, circulates more money, resolves more reliably, and is perfectly balanced under human control.** Per-board mean over ten games at matched seeds; dots are individual games. **Top-left:** GA winner 50 rounds vs. 131 default; four default games hit the 200-turn cap (red crosses). **Top-right:** \$66/round vs. \$39/round (target \$100). **Bottom-left:** default truncates 4/10, GA winner 0/10. **Bottom-right:** win-rate spread 0.60 default vs. **0.00** GA winner (5–5 split). N=10 per board, two players.

The GA winner finished in ∼50 rounds vs. 131 for the default (∼2.6× faster); four of ten default games hit the 200-turn cap with no winner, while all ten GA-winner games ended decisively. Money transfer was ∼68% higher (\$66 vs. \$39/round), and the win-rate spread was 0.00 (a 5–5 split) vs. 0.60 on the default.

Success criterion. The tool succeeds if, under human play, the agent-predicted effects (shorter, more decisive, livelier, fairer 2-player games) hold in the same direction and similar magnitude, and the feedback is *conservative* rather than optimistic. Both hold: all three evaluator classes (pool, LLM, human) agree on the direction *and* magnitude of every effect, and the humans showed a *larger* default-board failure than either simulator (40% draw rate vs. 7% in the pool and 0% under all-LLM seats), so the agent predictions under-state how broken the default board is for humans rather than over-state it.

The 0.00 fairness spread at N=10 is the strongest outcome possible at this sample size, and the pacing, money, and decisiveness effects are large enough to be unlikely artefacts of subject or seed. Given the limited sample, the full multi-subject playtest remains deferred (Appendix A.1). The key takeaway is that the method replicates real human play at the level of the optimised *metrics*: pacing, money flow, decisiveness, and fairness all move as predicted. It does not show that the method captures the subjective *feel* of the game; replicating felt experience and emotion, rather than measurable metrics, is an interesting direction for future work.

4.5 Game-design insights

System-level metric changes (Section 4.7), the per-archetype reshuffle (Section 4.6), and the human pilot (Section 4.4) together support a few concrete design insights that go beyond "the optimised board is better":

Expansion → solvency: the losing condition flips. On the default board (long games, low rents) the winner is usually whoever owns the most: rents are small relative to income, no single landing is fatal, and a long game amortises any acquisition, so accumulating property steadily pays off. The GA winner (∼60 rounds, ∼1.6× cost and ∼1.3× rent) inverts this: rents are now high enough that one bad landing on a developed cell can drain a $300 cash reserve, and the lap is too short to recover by slow accumulation. A player who spends down to build can be bankrupted by a single opponent landing, so cash discipline beats expansion: AggressiveBuilder's win-rate against CashHoarder collapses from 0.80 to 0.45 (Section 4.6). The losing condition changes from *owning too little* to *going bankrupt on a rent landing*.

Smaller colour groups make all-in builds work. On the default board a monopoly almost always needs three specific cells, which takes many laps or a trade to assemble, and the two 2-cell groups are edge cases: Brown is too cheap to win even fully built, Indigo too expensive to reach early. The GA winner's keep-mask shrinks most groups to 1–2 cells, so a group becomes a monopoly the moment its cells are bought, and the higher cost / rent multipliers make even a single developed cell lethal. Concentrated investment that is suicidal on the default (sinking cash into one property that cannot yet be built up) becomes a calculated risk here: max out houses and a hotel on one prime cell and a single opponent landing can end the game.

Blocking opponents stops working. The classic default-board tactic is denial: buy one cell of an opponent's three-cell set so they can never complete the monopoly, and the long game lets that denial compound. On the GA winner most sets are 1–2 cells, so an opponent completes them on the first landing and there is no "third cell" left to withhold; the higher property costs also make it unaffordable to buy spoiler properties and still develop your own. The lever disappears entirely. This is specifically a keep-mask effect: shrinking the board, not retuning rents, is what removes the blocking strategy.

Per-metric mechanisms. The single-objective ablations (Appendix A.4) isolate a specific board-design cause for each headline metric:

- **Game length** (∼60 vs. ∼100 rounds): a 12-cell lap leaves fewer turns between rent events.
- **Decisiveness** (draw rate 7%→2% under pool, 40%→0% under human play): higher base cost and rent force bankruptcy before the turn cap.
- **Money circulation** ($50→$73/round under pool): higher base rent on cells landed more often.
- **Fairness at 2p/3p** (spread 0.45→0.22 at 2p; −32% at 3p): Yellow dropped, most other groups shrunk to 1–2 cells, and per-property rents retuned to damp over-powered cells so no single group is structurally dominant, the same recipe the fairness-only ablation finds.

Why expert intuition is not enough. The broad direction here is within an expert's reach (raise rents and costs, drop properties to shorten the lap), but the balance is not. The GA tunes cost and rent *per-property* (0.6×–1.9×), damping specific over-powered cells rather than scaling uniformly, and the five objectives trade off, so pushing any one lever alone produces a lopsided board. The per-property cost and rent masks, and the lopsided single-objective optima, are shown in Figure 11 (Appendix A.4). Satisfying all five at once is the hard part, and it is what the search, not a hand-tuned prior, supplies.

4.6 Archetype reshuffle: *who* wins differently?

The composite averages over the 10 evaluation matchups, so it can fall even if only a subset of strategy pairs got worse. To see *which* archetypes the GA winner helps or hurts, we compute each strategy's mean win-rate against the rest of the pool ("win-rate vs. the field"). Figure 9 visualises the shift; the full per-archetype

numbers (all 10 named archetypes sorted by $\Delta = \mathrm{WR}_{\text{GA-2p}} - \mathrm{WR}_{\text{default}}$, plus the 20 sampled strategies) are in Table 3 (Appendix A.3).

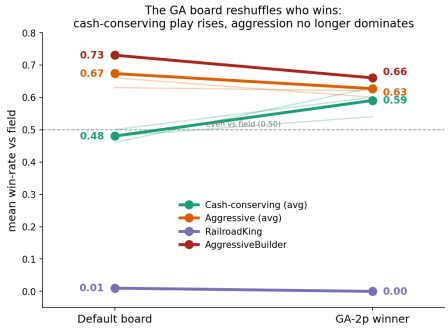


Figure 9: **The GA board reshuffles who wins.** Mean win-rate vs. the field, default board → GA-2p winner. Cash-conserving archetypes (CashHoarder, RiskAverse, Passive) rise from middling to top-tier while the aggressive group declines; RailroadKing (bottom) and AggressiveBuilder (top) stay board-robust on both boards.

The board change does *not* flatten win-rates uniformly; it reshuffles which archetypes are top-tier. Slow, cash-conserving archetypes move from middling to top-tier (CashHoarder +0.18, RiskAverse +0.10, Passive +0.07, LowCostOnly +0.06), while HighCostOnly collapses (−0.18) because the keep-mask prunes the expensive groups it relies on. This is the same expansion-to-solvency flip described in Section 4.5: on the short, high-rent GA board cash discipline beats expansion, so AggressiveBuilder's win-rate against CashHoarder drops from 0.80 to 0.45. With the agents held fixed, board design alone is enough to change which playstyles win and to level the field, narrowing the win-rate gap between archetypes and making the game fairer. This inverts the naive prior that shorter games reward aggression, here they reward solvency, and the effect is only visible against the diverse pool, not against a single expert's playstyle.

The reshuffle is concentrated in the *middle* of the distribution: the top-3 default strategies (AggressiveBuilder, Balanced, Trader, all WR ≥ 0.66) stay top-3 on the GA winner, while RailroadKing, which ignores every colour group, stays pinned near zero on both boards. Top and bottom are board-robust, consistent with the winner being a mutation of the default rather than a board built from scratch; some pairs are even immutable, e.g. *Trader* vs. *RailroadKing* stays 100%/0% under every cost, rent, and keep-mask setting. The lesson is twofold: archetype performance is a property of the *(strategy, board)* pair, so evaluating a strategy across boards reveals whether it is robustly good rather than good on one board; and board design, though a strong lever for system-level properties (length, draws, money, fairness), cannot remove agent-level dominance that is structural to the pool, which needs a strategy-side fix (rule changes, matchmaking, or pool curation).

4.7 Cross-player-count evaluation (2–8 players)

We cross-evaluate the 2p- and 3p-trained GA winners across all seven player counts from 2 to 8 at n=1000 games per cell (Table 2, Figure 10). To compare across counts we normalise money flow to *transfer per player-turn* (\$/round divided by n, target 50) and use the matching cross-count-comparable composite (Section 3).

The headline finding is that *game length, decisiveness, money transfer, and the composite advantage all generalise from the 2p/3p training regime to every player count from* 2 *to* 8*, while fairness is strongly dependent on the number of players* (Figure 10). The optimised boards stay near the target 60 rounds at every count while the default climbs from 104 rounds at 2p to 158 at 8p, and they resolve far more reliably: the default fails to finish 71% of games at 8p, against at most 22% for the GA winners (12% for the GA-3p winner). Both winners keep roughly a 2× composite advantage over the default at every count, and transfer per player-turn rises toward the target 50 (GA-2p winner 37→57) while the default plateaus near 32.

Fairness is the one axis that does not transfer cleanly. Within the training regimes (2p/3p) the GA winners are 30−53% fairer than the default, but from 4p onward that advantage erodes and inverts, reaching 69−101% *less* fair at 7−8p. Two effects drive this: the fairness metric $\overline{F}$ mechanically compresses toward

Design	Eval	score ↓	$\overline{F}$ ↓	$F_{\max}$ ↓	rounds (→ 60)	draw % ↓	transfer/turn (→ 50)
default board	2p	1.463	0.454	0.940	103.9	7.4	24.8
default board	3p	1.329	0.412	0.680	110.8	17.6	33.1
default board	4p	1.385	**0.309**	0.640	125.6	38.3	34.3
default board	5p	1.434	0.269	0.680	131.0	46.9	34.6
default board	6p	1.292	**0.181**	**0.340**	140.0	57.0	32.8
default board	7p	1.376	**0.152**	**0.330**	150.7	65.0	31.9
default board	8p	1.401	**0.117**	**0.290**	157.6	71.4	31.4
GA-2p winner	2p	**0.774**	**0.215**	**0.910**	**62.2**	1.7	**36.7**
GA-2p winner	3p	0.729	**0.273**	0.730	65.7	3.3	**44.4**
GA-2p winner	4p	**0.646**	0.310	0.580	63.4	3.9	**49.0**
GA-2p winner	5p	**0.494**	**0.235**	**0.420**	**62.7**	5.4	51.8
GA-2p winner	6p	0.535	0.242	0.420	**64.3**	8.1	**53.8**
GA-2p winner	7p	0.572	0.237	0.400	66.8	14.3	**55.8**
GA-2p winner	8p	0.548	0.198	0.360	67.5	21.6	**57.1**
GA-3p winner	2p	0.897	0.287	0.970	56.2	**0.5**	34.7
GA-3p winner	3p	**0.602**	0.280	**0.490**	**56.3**	**0.6**	42.6
GA-3p winner	4p	0.656	0.336	**0.550**	**56.7**	**1.8**	48.0
GA-3p winner	5p	0.603	0.314	0.460	54.8	**1.8**	**51.7**
GA-3p winner	6p	**0.522**	0.264	0.360	55.4	**4.4**	54.5
GA-3p winner	7p	**0.559**	0.281	0.360	**55.8**	**7.0**	57.1
GA-3p winner	8p	**0.514**	0.235	0.320	**56.7**	**12.4**	59.1

Table 2: **Cross-player-count evaluation at n=1000 games per cell across 2–8 players, 66-dim keep-mask design space.** All seat permutations balanced via cyclic rotations (extended to n = 4–8). Header arrows: ↓ means lower is better; (→T) means the target value is T and deviation in either direction is penalised. **Bold** marks the best (lowest) value in each column among the three designs at that player count; underline marks the second-best. The **score** column is the cross-count-comparable composite (transfer term against \$/player-turn vs. target 50); **transfer/turn** = (\$/round) / n_{players} (target 50), removing the trivial linear scaling of money flow with seats.

zero as seats grow (the per-seat baseline win rate is $1/n$), which exaggerates the default's gain at high counts; and the GA balanced the board specifically for 2p/3p play, so that balance no longer holds once more players are added. The takeaway is that fairness, unlike pacing, money, and decisiveness, is *strongly dependent on the number of players*: it is a regime-specific property and must be optimised for the count at which the game is actually played.

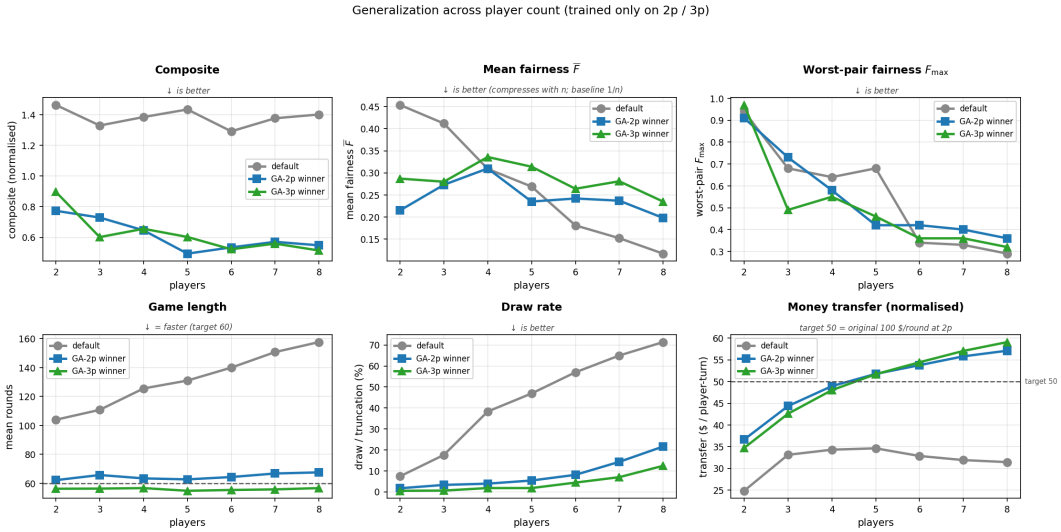


Figure 10: **Generalisation across player count, trained only on 2*p*/3*p*.** The six Table 2 metrics across 2–8 players for the default board (gray) and the two GA winners (blue, green). The optimised boards stay near the target 60 rounds, below 25% draws, and keep a clean composite advantage at every count; mean fairness $\overline{F}$ is the only axis that does not transfer (winners fairer in-distribution, less fair at high counts, partly because $\overline{F}$ compresses toward 0 as seats grow).

4.8 Limitations

The main caveats: the 17-dim strategy space is broad but not exhaustive; the N=10 pilot rules out idiosyncrasy in trained-effect direction but not in the broader population; Qwen2.5-1.5B is the only LLM in the study; and the design space covers only per-property cost/rent multipliers and the keep-mask on the standard 40-cell US Monopoly layout, so other design levers, such as the starting salary, dice modifications, entirely new properties, and new board interconnections or topology, are left unexplored. Full discussion in Appendix A.2.

4.9 Conclusion

The goal was to automate the inner loop of beta-testing, replacing many human testers across many sessions with a population of cheap, imperfect simulated players. Our results show this works: three independent validations clear the success bar. **(i) Generalisation:** the 2p/3p-trained GA winners beat the default at every count from 2 to 8, playing ∼2–3× faster, finishing reliably, and holding a ∼2× composite advantage, with fairness the one regime-specific exception. **(ii) Cross-class agreement:** the pool-driven board beats an LLM-trained design on 4 of 5 stable metrics under *both* evaluator classes, so a diverse rule-based pool generalises to an unseen LLM evaluator at zero per-decision cost. **(iii) Human pilot:** an N=10 two-player playtest confirms the direction and magnitude of every trained effect, with humans showing an even *larger* default-board failure than the simulators predicted (conservative, not optimistic).

Beyond "the optimised board is better", the framework lets us analyse *both* board characteristics and player behaviour: the GA winner flips the losing condition from owning too little to going bankrupt on a rent landing, which reshuffles which archetypes win, neutralises blocking, and makes small-group all-in builds viable. As Sections 4.5 and 4.6 detail, expert intuition recovers the broad direction (raise rents and costs, drop properties) but not the per-property balance, the multi-objective trade-off, or the counterintuitive inversion of which playstyle wins, none of which can even be verified without a diverse agent population: the search supplies the precise configuration and the pool supplies the verification. More broadly, the recipe (closed-loop GA + diverse evaluator pool + independent cross-class checks) transfers to any parameterised multi-agent system: market mechanisms, RL environments, or recommender systems. In practice another designer supplies their own objective and a deliberately diverse pool of cheap agents, searches their design space, and trusts a candidate only once it survives an independent cross-class check, getting fast, repeatable design feedback long before spending scarce human-testing time.

5 Author contributions

Selim Emir Can. Project idea, implementation, experiments, figures, and writing; all work in this report is mine.

A Implementation details

This appendix collects implementation details that are useful for reproducibility but would clutter the main text. All source paths are relative to `monopoly/` on the project repository.

A.1 Full playtest (deferred): board shortlist

Beyond the N=10 pilot reported in Section 4.4, we did not run the full multi-subject playtest within the project window. Rather than report a synthetic surrogate, we publish the shortlist of boards we would playtest, with the design hypothesis each board is intended to probe. The shortlist is small (4 boards) so a pilot is feasible in a single 60–90 min session, and each board contrasts a different design lever against the default, so that even a handful of subjects gives readable signal on which axis humans actually respond to.

Boards.

1. **Default Monopoly** (control). Unperturbed `settings.py` board. Baseline for every subjective rating.

2. **GA-2p combined winner** (Section 4.1). Best composite-score 2p board: shorter, lower draw rate, and more inter-player money transfer than default. Tests *whether the combined objective tracks perceived game quality*, the core falsifiable claim of the GA half of the project.
3. **GA-3p combined winner** (Section 4.7). Cross-evaluation (Table 2) shows the 2p and 3p winners are structurally different boards; this comparison tests *whether human players can also feel the 2p↔3p design split*, or whether the difference is only legible to the rule-based pool.
4. **Money-only 2p optimum** (Appendix A.4). Board produced by setting $w_{\text{money}}=1$ and zeroing every other weight, pushing per-round inter-player cash transfer from ≈ 50 \$/r (default) toward the target 100 \$/r while making no claim about fairness or pacing. Tests *whether interactivity* per se*, independent of fairness and length, drives perceived engagement.* This is the most novel of the four boards, because money-transfer rate is the only objective term the project introduces that has no direct precedent in the search-based game-design literature cited in Section 2.

Why these four, and not the other ablation winners. The shortlist deliberately excludes *length-only*, *draw-only*, and *fairness-only* optima. Length and draw rate also change in the combined-objective winner (board 2), so a playtest on those single-axis boards would not separate the hypothesis "length matters" from "the combined objective matters." Fairness is a property of the win-rate distribution across many games and is not directly observable from a single session: a single playtest game tells a subject who happened to win, not whether the board is fair. Money-only, by contrast, manipulates an axis that should be *felt within a single game* (more trades, more rent events, more state change per turn), so it is the only ablation winner where a small pilot would carry useful signal.

What we would measure. For each board: post-game Likert ratings (1–5) on pacing, fairness, engagement, and overall enjoyment, plus a forced-choice "would you play this version again." Between-board comparisons would be non-parametric (Wilcoxon signed-rank, Bonferroni-corrected across the 6 board pairs); per-board scores reported with bootstrap 95% CIs. The minimum useful pilot is $N=8$ subjects, each playing all four boards in counterbalanced order (Latin square on the 2p boards; the 3p board played in groups of three).

What this shortlist would falsify. The two strongest claims the GA-side of the report makes are (a) the combined objective produces a measurably different board, and (b) the per-objective ablations expose orthogonal design axes. A playtest where board 2 feels the same as board 1, or board 4 scores the same as board 1 on engagement, would mean the matching claim above is wrong. We frame the shortlist as a *falsification plan*, not a confirmation plan.

A.2 Limitations

- **Strategy-pool coverage.** The 17-dim parametric space is broad but not exhaustive, and human players behave outside it. The 20 sampled strategies broaden the pool beyond the 10 archetypes and the sampling procedure is documented and reproducible, but a real deployment would want to learn the strategy distribution from logs.
- **Human playtest is a small pilot, not a full study.** Only an $N=10$ two-player pilot was run (Section 4.4): two players played the default and the GA-2p combined winner under ten matched seeds, which confirms all four trained effects (pacing, money circulation, decisiveness, and win-balance) on this single subject pair but cannot rule out idiosyncrasy in playing style across the broader population. The full multi-subject playtest remains deferred; the shortlist in Appendix A.1 curates 4 boards and names the design hypothesis each is intended to probe, with a proposed pilot protocol ($N=8$, counterbalanced, Likert plus forced-choice).
- **Fairness is strongly dependent on the number of players.** The pacing, decisiveness, and composite advantages generalise cleanly from the 2p / 3p training regime to every count from 2 to 8 (Section 4.7, Figure 10), but fairness does not: the GA winners are $30-53\%$ fairer than the default at training counts and $69-101\%$ *less* fair at $7-8$p. A deployment at a player count outside the training regime should re-optimise the fairness term against that count, or accept that fairness gains may not carry over.

- **LLM scale.** Qwen2.5-1.5B is the only LLM in the study. Larger models or different prompt styles may change the agreement profile. Greedy decoding does, however, make the probe deterministic.
- **Single-board scope.** All optimisation runs on the standard 40-cell US Monopoly layout (with structural shrinkage via keep-mask). Other game families would require a new design-space encoder, but the rest of the pipeline (objective, pool, GA, validator) carries over unchanged.

A.3 Per-archetype win-rate table

Full per-archetype win-rate-vs-the-field for the default board and the GA-2p winner, the numbers behind Figure 9.

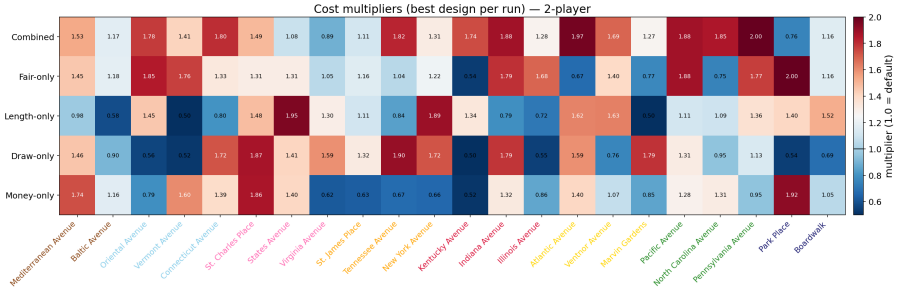
Archetype	WR(default)	WR(GA-2p)	Δ	Identity
CashHoarder	0.46	0.63	**+0.18**	$1k+ unspendable, no trades
RiskAverse	0.50	0.60	+0.10	hoard, skip expensive groups
Passive	0.48	0.54	+0.07	no trades, no rails/utils
LowCostOnly	0.56	0.62	+0.06	skip 5 expensive groups
Bully	0.63	0.62	−0.00	ultra-aggressive build + trade
RailroadKing	0.01	0.00	−0.01	rails only, ignore colour groups
Trader	0.66	0.60	−0.05	aggressive build + trade
AggressiveBuilder	**0.73**	**0.66**	−0.06	build at zero floor, no trades
Balanced	0.71	0.64	−0.06	balanced thresholds, trades
HighCostOnly	0.56	0.38	**−0.18**	skip 3 cheap groups
Random sampled (n=20)	0.45	0.48	+0.03	20 random draws from the parameter space

Table 3: **Per-archetype win-rate against the field, default vs. GA-2p winner.** WR vs. field = mean win-rate against the other 29 pool strategies, 20 games per ordered pair, 580 games/archetype/board. Sorted by Δ descending. Largest positive and negative deltas in bold.

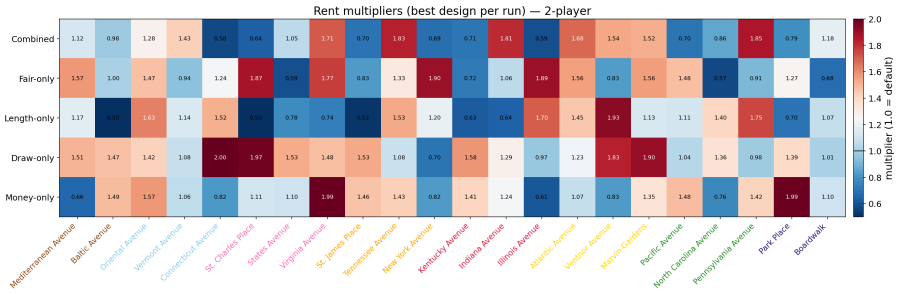
A.4 Single-objective ablations

We ran five ablations per player count, each setting one weight to 1 and the others to 0 (*fairness-only, length-only, draw-only, money-only, combined*), and reported best-so-far convergence and the full metric tuple at the optimum. Each ablation reaches a different optimum: the length-only run drives $\overline{R}$ to within ≈ 1 round of $R^* = 60$ but degrades fairness, while the fairness-only run nearly halves $\overline{F}$ but inflates draws. The per-aspect heatmaps in Figure 11 show the ablations carving visibly different shapes in the cost-multiplier, rent-multiplier, and keep-mask sub-spaces. The composite is therefore non-redundant: if any two terms drove the optimiser toward the same board, the corresponding ablations would have near-identical metrics, which they do not.

The same finding is visible at the level of physical boards: the five winners (combined + four ablations) are five visibly different shapes, with different properties kept or dropped and different per-property cost / rent multipliers.



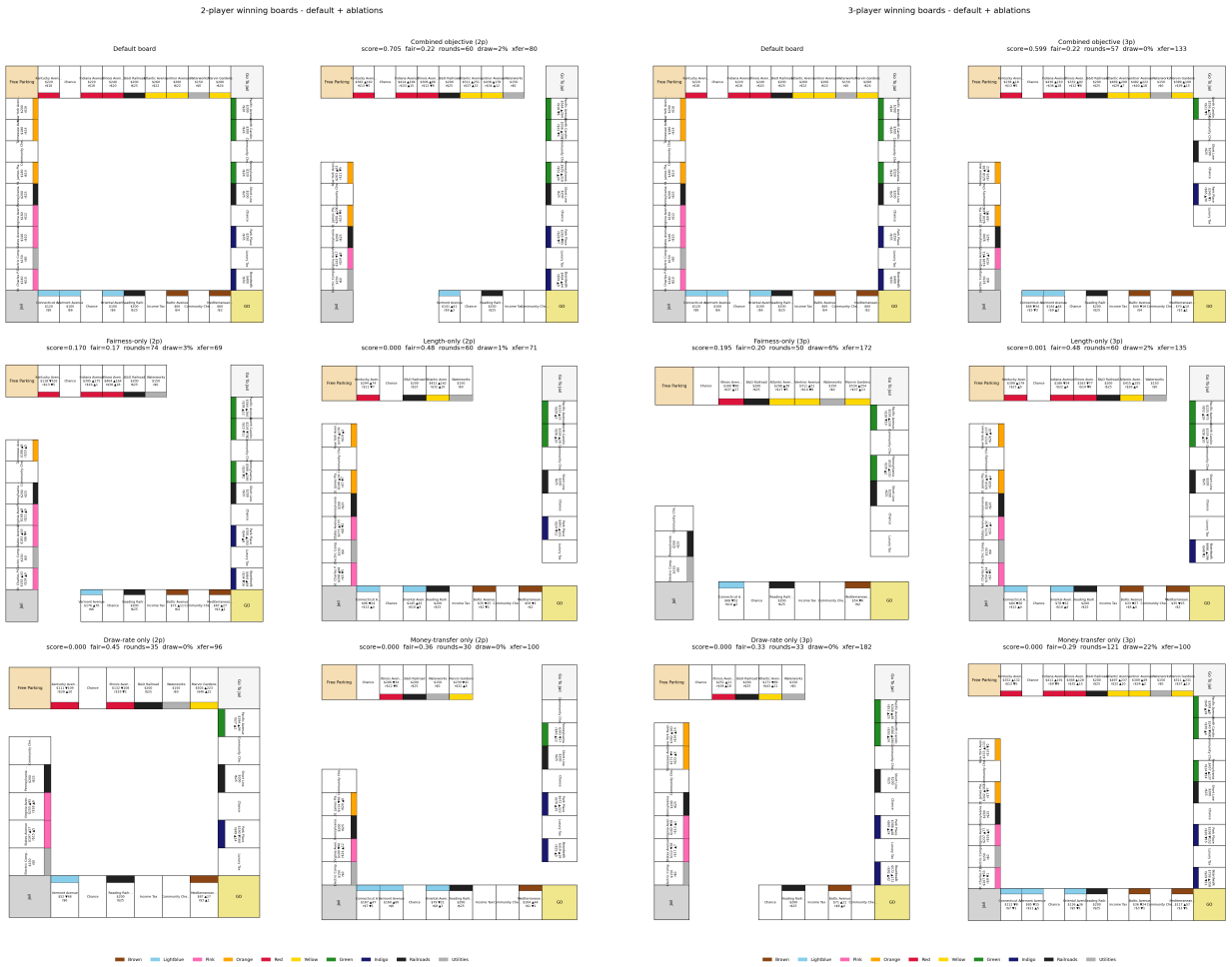
(a) cost mult. (2p)



(b) rent mult. (2p)

(c) keep-mask (2p)

Figure 11: **Each ablation carves a different shape in design space (2p).** Per-property cost multiplier (a), rent multiplier (b), and keep-mask bit (c) at the optimum of each GA run. Within each panel, *rows* are the five ablations (*combined, fair-only, length-only, draw-only, money-only*, top to bottom) and *columns* are the 22 ownable properties in board order (Mediterranean Avenue at left through Boardwalk at right). Cell values are the multiplier at the optimum (cost/rent panels, 0.5 to 2.0, with 1.0 being the default identity) or the keep-bit (keep-mask panel: green = kept, black = dropped). If any two ablations drove the optimiser to the same board, those rows would be near-identical; the visible inter-row variation is the empirical evidence that the composite objective is non-redundant.



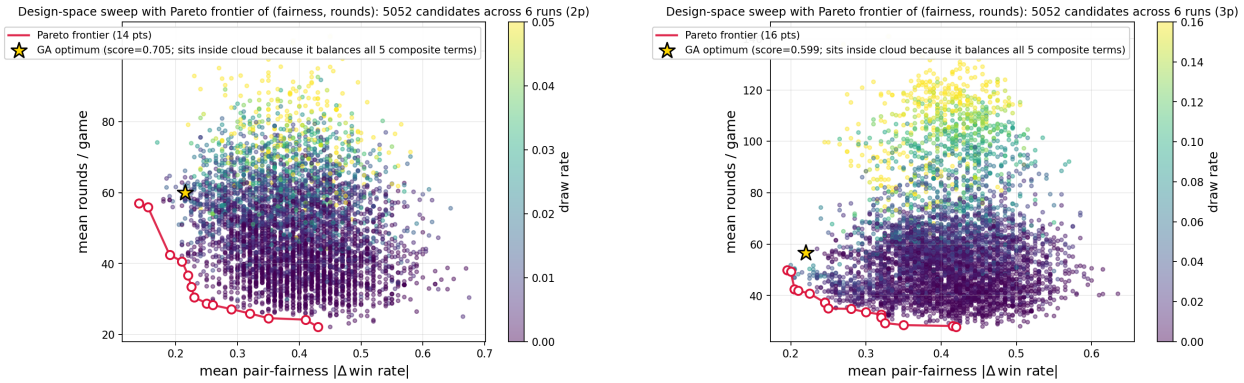
(a) 2 players

(b) 3 players

Figure 12: **Each ablation produces a visibly different physical board.** Six panels per player count: default (top-left) and the five GA-run winners (combined, fair-only, length-only, draw-only, money-only). Same non-redundancy finding as Figure 11, made spatial.

A.5 Pareto trade-off surface

The (fairness, rounds) trade-off surface (Figure 13) visualises the design-space geometry that the cross-player-count evaluation (Section 4.7) reads off. The two clouds have visibly distinct shapes, which is what makes that cross-evaluation finding a property of the design space rather than seed noise.



(a) 2p (b) 3p

Figure 13: **The (fairness, rounds) trade-off surface and its Pareto frontier.** Each marker is one of 5,052 candidates evaluated across all six search runs (random + GA + four single-objective ablations) on the 66-dim keep-mask design space; colour encodes draw rate. The crimson line traces the actual 2D Pareto frontier of (fairness, rounds), both minimised: 14 points at 2p, 16 at 3p, with a clean knee around fairness 0.22 and rounds 30−40 at 2p (a slightly higher knee at 3p). The gold star is the combined-objective GA optimum; it sits *inside* the cloud rather than on the 2D frontier because the composite weights five terms (fairness, max-fairness, length, draw, transfer), not just two; the GA's optimum is on the 5D Pareto surface, of which the (fairness, rounds) projection is a shadow. The 2p sweep reaches mean rounds as low as ~ 22 (the keep-mask shrinks the lap aggressively), while the 3p frontier bottoms out closer to ~ 35 rounds; the two clouds have visibly different shapes, which is what makes the cross-evaluation finding (Table 2) a property of the design space rather than seed noise.

A.6 Training-vs-deployment overfit

A secondary finding worth recording: *optimisation-set overfit is small on both designs.* At the n=200 optimisation budget the GA-2p winner scored 0.705 on its training matchup set; at n=1000 deployment it scores 0.773, an overfit of +0.068 (Table 4). The GA-3p winner scored 0.599 at training and 0.641 at deployment, an overfit of +0.042. Both gaps are small ($\leq 9\%$ of the deployment score) relative to the headline $\sim$47% improvement over the default, so the directional improvement is robust to re-evaluation at higher n.

Design	training (n=200) ↓	deployment (n=1000) ↓	overfit ↓
GA-2p winner	0.705	0.773	+0.068
GA-3p winner	0.599	0.641	+0.042

Table 4: **Optimisation-set vs. deployment-set composite score, on each design's own training regime.** Header arrows: ↓ means lower is better. Both designs show small positive overfit (+0.068 and +0.042 respectively, both $<10\%$ of the deployment score) relative to the headline $\sim$47% improvement over the default. Training number is the best score in the GA's own `evals.jsonl`; deployment number is the corresponding row of Table 2.

A.7 LLMPlayer prompt and validator

Model and decoding. Qwen2.5-1.5B-Instruct [7] loaded with the `transformers` library, run with greedy decoding (`do_sample=False`), `max_new_tokens=160`, and additional EOS tokens `<|im_end|>` and `<|endoftext|>`. The same model and decoding settings are used for all four runs (LLM-only evaluation and the LLM-driven GA, at 2p and 3p).

Conversation structure. Each LLM buy decision is a six-message conversation: 1 system message, 4 few-shot user/assistant exchanges (2 *BUY* and 2 *PASS* demonstrations), and 1 runtime user message describing the actual decision. The full transcript with all four few-shot examples is checked into the repository at `prompts/llm_player_prompts.txt`.

System prompt (v2, verbatim).

> *You are a strategic Monopoly player who decides whether to buy properties. Your goal is to win by bankrupting opponents through rent. You must NOT mindlessly buy every property: passing is correct when (a) cash is dangerously low, (b) the property is in a group an opponent already dominates and you have no path to monopoly, or (c) the rent return is poor relative to the cost. Equally, buying is correct when the property advances a group you already own most of, or when it denies the group to an opponent who would otherwise complete it.*
>
> *You receive a STATE block with 10 fields. Before reasoning, you MUST emit an ECHO block that copies all 10 STATE fields verbatim (numeric values must equal STATE; string values must equal STATE). The ECHO block is checked deterministically – if it differs from STATE in any field you will be asked to redo the answer. Only after the ECHO block do you write REASON and ANSWER. Do not invent any number or name not present in STATE. A purchase ONLY "completes a monopoly" when `you_own_in_group + 1 == group_size`; do not claim completion otherwise.*
>
> *Reply in EXACTLY this format and order: `ECHO:` followed by the 10 STATE fields (`cash`, `property`, `group`, `cost`, `base_rent`, `you_own_total`, `you_own_in_group`, `group_size`, `opp_own_in_group`, `opp_own_total`); then `REASON: <one short sentence citing only STATE numbers>`; then `ANSWER: BUY` or `ANSWER: PASS`. Do not output anything else.*

v1 vs. v2 prompts. The v1 prompt was free-form: it asked for "BUY" or "PASS" with a one-sentence reason but did not require the ECHO copy of STATE. The v1 validator parsed the free-form response with a regex and was prone to a now-fixed integer/float type mismatch on cash (Section 4.2). v2 added (i) the verbatim ECHO requirement, (ii) deterministic per-field equality checking on the echoed values, and (iii) a corrective retry loop that replays prior assistant turns plus a follow-up user message naming the mismatched fields. Both versions share the same four few-shot examples and the same STATE format.

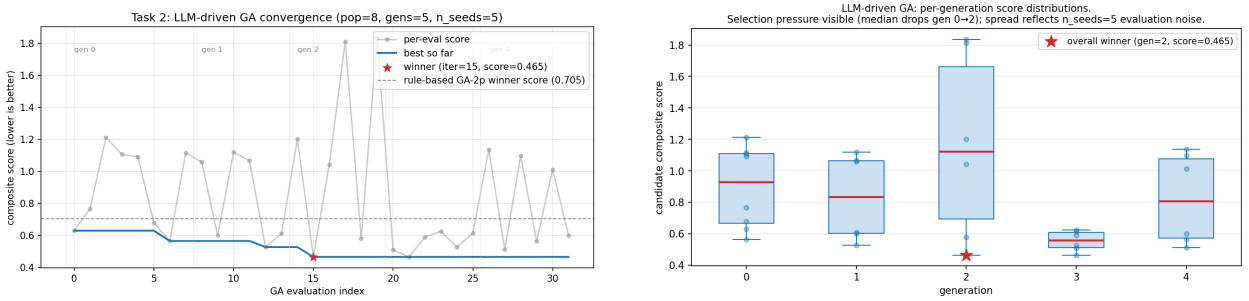
Echo validator. `LLMPlayer._check_echo` parses the ECHO block with a multiline regex and equality-checks each of the 10 fields against ground-truth state, with a \$0.50 tolerance on cash to absorb float drift from rent multipliers (the original fixed-int validator caused 100 spurious flags in v1).

Retry mechanism. On any echo mismatch, the player retries up to `MAX_RETRIES = 4` times. Each retry replays prior assistant turns plus a corrective user message that names the mismatched fields. The final attempt's parsed answer is the decision used by the simulator, even if it still has echo issues; the per-decision JSONL log records every attempt for post-hoc analysis.

Pre-filter. Affordability and cash-floor checks short-circuit the LLM call when the decision is forced (cash $<$ cost forces PASS, etc.). This eliminates 60–80% of would-be calls and keeps the LLM evaluation pipeline within practical wall-clock budgets.

A.8 LLM-driven GA convergence

The LLM-driven GA loop (Section 4.3) converges over its 32 evaluations and produces a winning board that beats the default under all-LLM evaluation; whether that board is a real design optimum or only an LLM-evaluator optimum is what Figure 6 resolves in the main text. The convergence curves and per-generation score distribution are below.



(a) LLM-GA best-so-far

(b) Score distribution

Figure 14: **An LLM-only evaluator can drive the same GA loop. (a)** Per-evaluation composite score (grey) and best-so-far (blue) over the GA's 32 evaluations (population 8, 5 generations, n_{seeds}=5, with elitism keeping the unique evaluation count at 32). The dashed reference is the rule-based GA-2p winner's score under the same all-LLM evaluator (0.728): the LLM-driven loop finds a board (red star, iter 15, score 0.465) that beats even the rule-based winner when scored by an LLM. **(b)** Per-generation distribution of candidate scores; the median visibly drops from generation 0 to generation 2 (selection pressure), and the spread within each generation reflects the n_{seeds}=5 evaluation noise. Total wall time 4h 58min on a single 12 GB GPU.

A.9 LLM buy-rate diagnostics

To verify that the LLM probe is making rational rather than uniform-random decisions, we slice its buy rate by available cash and by monopoly-completion situation on both boards. The probe buys monotonically more often as cash rises, saturates near 1.00 when it already partially owns the colour group, and drops to ∼0.13 when the opponent already dominates the group; the directional responses are essentially identical on both boards (Figure 15).

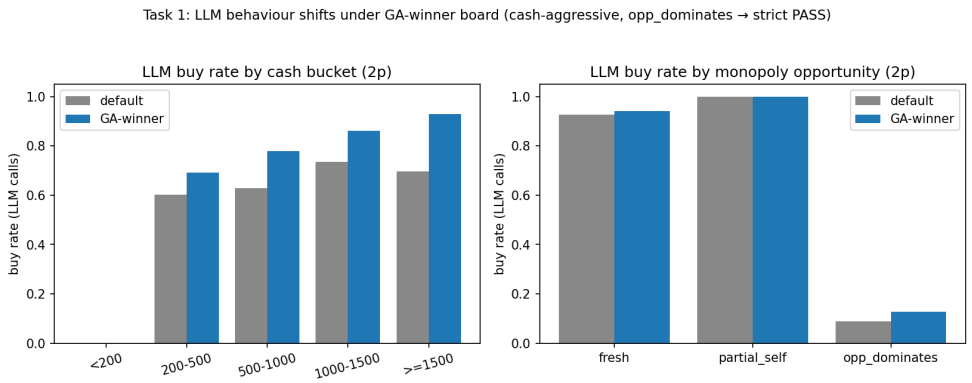


Figure 15: **LLM buy-rate slices, default board vs. GA-2p winner. (a) By cash bucket.** Buy rate conditioned on the LLM's available cash at decision time. The rate rises monotonically with cash, the rational ordering for any non-degenerate strategy; the empty < 200 bucket reflects the affordability pre-filter that short-circuits the LLM call when the player cannot buy anyway. On the GA-winner board the LLM commits its cash hoard more aggressively in the ≥ 1500 bucket (~ 0.93 vs. ~ 0.70 on the default), so the optimised board successfully pulls behaviour out of the cash-hoarding regime. **(b) By monopoly-completion situation.** *fresh*: no player owns any property in the landed colour group, so a purchase starts a new collection. *partial_self*: the LLM already owns one or more properties in the group and the opponent does not dominate it. *opp_dominates*: the opponent owns at least half the group and at least as many properties as the LLM, so buying mostly funds their next rent payment. Buy rates respond rationally across all three: ~ 1.00 on *partial_self* (saturated buy), ~ 0.94 on *fresh*, and ~ 0.13 on *opp_dominates* (strict PASS). The probe is not behaving as a uniform random oracle, and the directional responses are essentially identical on both boards.

A.10 v1-vs-v2 hallucination audit

The cross-class probe (Section 4.2) ran two prompt versions: v1 (free-form BUY/PASS with a one-sentence reason) and v2 (the structured `STATE/ECHO/REASON/ANSWER` prompt described above). v1's validator had a subtle integer/float type mismatch on the cash field, and retries against a wrong validator induced *more* plausible-looking confabulation rather than less, because the model was being told a correct value was wrong and produced a different "corrected" answer to satisfy the validator. We document this here rather than tuning the validator until it disappears: a protocol whose failure modes are reported alongside its results is auditable; one whose are not is a benchmark trick. v2 fixes the validator and adds the verbatim ECHO copy; across 2,207 calls under v2 the model never fabricates state at first-pass and the validator never falsely flags a correct output.

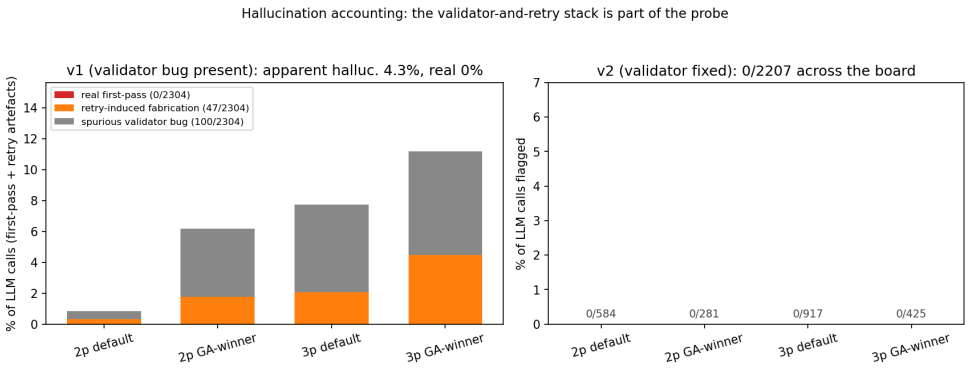


Figure 16: **The validator-and-retry stack is part of the probe: a faulty validator does not just produce noise, it pulls the model toward fabrication.** Per-call validation outcomes for the v1 prompt (left, with a known validator bug) and the v2 prompt (right, bug fixed). Each bar is stacked into three categories: *real first-pass* hallucinations (the model fabricated state on its first attempt), *retry-induced fabrication* (the model's first-pass output was correct but the buggy validator flagged it, and the model produced fabricated state on retry to "satisfy" the flag), and *spurious validator bug* (the validator falsely flagged correct outputs that the model did not change). On v1 across 2,304 calls, real first-pass = 0, retry-induced = 47, spurious = 100; the model itself never hallucinated. v2 drives all three categories to zero across all 2,207 calls. The reportable second-order finding: validating against a wrong ground truth is worse than not validating, because retries can teach the model to produce the "correct" wrong answer.

A.11 Evaluation methodology

Three standard techniques underpin the evaluation. Shared random numbers across candidates [4] reduce variance, so far fewer games per candidate are needed than uncorrelated evaluation would require, and an unchanged design vector yields byte-identical metrics. Win-rate uncertainty is quoted with Wilson 95% confidence intervals [11] (about ±3pp at the n=1000 deployment evaluation), and per-objective single-knob ablations isolate each composite term. All three are standard; the only novelty is connecting them to the agent-as-design-probe framing.

A.12 Pipeline components and driver scripts

Optimisation stack. The optimisation stack lives under `monopoly/optimizer/` and `monopoly/scripts/`. `ParametricPlayer` together with `ParametricPlayerSettings` subsumes every existing rule-based subclass under a single 17-knob agent (5 continuous, 4 boolean, 8-bit colour-group mask). `strategy_pool.py` produces the 30-strategy pool described in Section 1, deterministic via a fixed seed and saved once for reuse across runs. `design_space.py` maps a 66-dim vector to a fresh `GameConfig` by applying per-property multipliers to `cost_base`, `cost_house`, `rent_base`, and `rent_house`, and physically removing properties whose keep-bit is zero (structurally shortening the lap rather than substituting `FreeParking`); the identity vector is the default board. `simulate.py` is a thin re-implementation of the inner game loop that returns a per-game stat dict (winner, rounds, bankruptcy map, inter-player transfer total) instead of only logging; inter-player cash flow is captured via a context-manager monkey-patch of `Player.pay_money`, with non-player payments (taxes, fines, salary) excluded, and trade calls are capped at 5 per (player, turn) to break an unbounded loop in upstream code. `search.py` implements both random search and a GA (population 20, 10 generations, tournament selection k=3, uniform crossover, σ=0.1 Gaussian mutation on continuous dims, 1-bit-flip mutation on the keep-mask, elitism 2); both produce identical JSONL.

Driver scripts. The end-to-end pipeline is driven by five scripts. `scripts/optimize_board.py` runs the rule-based search and emits one JSONL line per evaluation. `scripts/cross_eval.py` runs designs through both player-count harnesses at n=1000. `scripts/strategy_heatmap.py` produces 30×30 strategy-vs-strategy win-rate matrices on any chosen design. `scripts/eval_llm_on_boards.py` runs the all-LLM-seat

probe with per-decision logging, and `scripts/optimize_board_llm.py` re-runs the GA with the LLM in the evaluator role.

A.13 GA, ablations, and reproducibility

Search hyperparameters (canonical protocol, logs in `logs/optimizer_v3/`).

- GA: `--pop 30 --generations 30 --elitism 2`, tournament selection k=3, uniform crossover, Gaussian mutation σ=0.1 on continuous dims, 1-bit-flip mutation on the keep-mask. Total evaluations: $30+29\times28 = 842$.
- Random search: `--iters 842` for matched evaluation budget.
- Per-candidate evaluation: `--n-games 200`, distributed as 10 fixed matchups × 20 games per matchup, with seeds shared across all candidates.
- Composite weights $w = (1, 0.5, 0.5, 0.3, 0.3)$ for $(\overline{F}, F_{\max}, |\overline{R}-R^*|/R^*, \overline{D}, |\overline{T}-T^*|/T^*)$ with targets $R^* = 60$ rounds, $T^* = 100$ \$/round.
- Single-objective ablations set the chosen weight to 1 and the others to 0; same search hyperparameters otherwise.
- Cross-evaluation at n=1000 games per cell (deployment table) uses the same matchup seeds.

Compute. Hardware: a single CPU at ≈ 6 ms/game for the rule-based stack; a single 11 GB GPU locally for LLM games at ≈ 6–10 s/decision. Total games for the entire study: ≈ 165k for the rule-based pipeline, ≈ 2300 for the LLM cross-class probe, and ≈ 2000 for the LLM-driven GA and its cross-evaluation.

Reproducibility.

- All seeds (`--base-seed=42`, `--search-seed=0`, `--matchup-seed=1234`) are recorded in each run's `<run_name>.meta.json`.
- Re-running any `(design vector, matchups, seeds)` triple yields byte-identical metrics. We verified this by diffing two re-runs of the same config.
- Cross-player-count scaling table data (Table 2) is in `logs/optimizer_v3/cross_eval_scaling.json`; the 2p / 3p columns reproduce the original `logs/optimizer_v3/cross_eval_mask.json` bit-for-bit.
- Trigger scripts: `scripts/rerun_strategy_experiments_overnight.bat` (rule-based, ∼2.5h CPU) and `scripts/rerun_llm_phase_c_v3.bat` (LLM-only evaluation re-run, ∼6h on a 12 GB GPU).

References

[1] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

[2] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning.* Addison-Wesley, 1989.

[3] Christoffer Holmgård, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Evolving personas for player decision modeling. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.

[4] Averill M. Law and W. David Kelton. *Simulation Modeling and Analysis.* McGraw-Hill, 5th edition, 2014. Chapter on Common Random Numbers (CRN) for variance reduction in stochastic simulation.

[5] Antonios Liapis, Georgios N. Yannakakis, and Julian Togelius. Sentient sketchbook: Computer-aided game level authoring. In *Proceedings of the 8th International Conference on the Foundations of Digital Games (FDG)*, pages 213–220, 2013.

[6] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22, 2023.

[7] Qwen Team. Qwen2.5 technical report. Technical report, Alibaba Group, 2024.

[8] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Computational Synthesis and Creative Systems. Springer, 2016.

[9] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 34, 2021.

[10] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[11] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.